

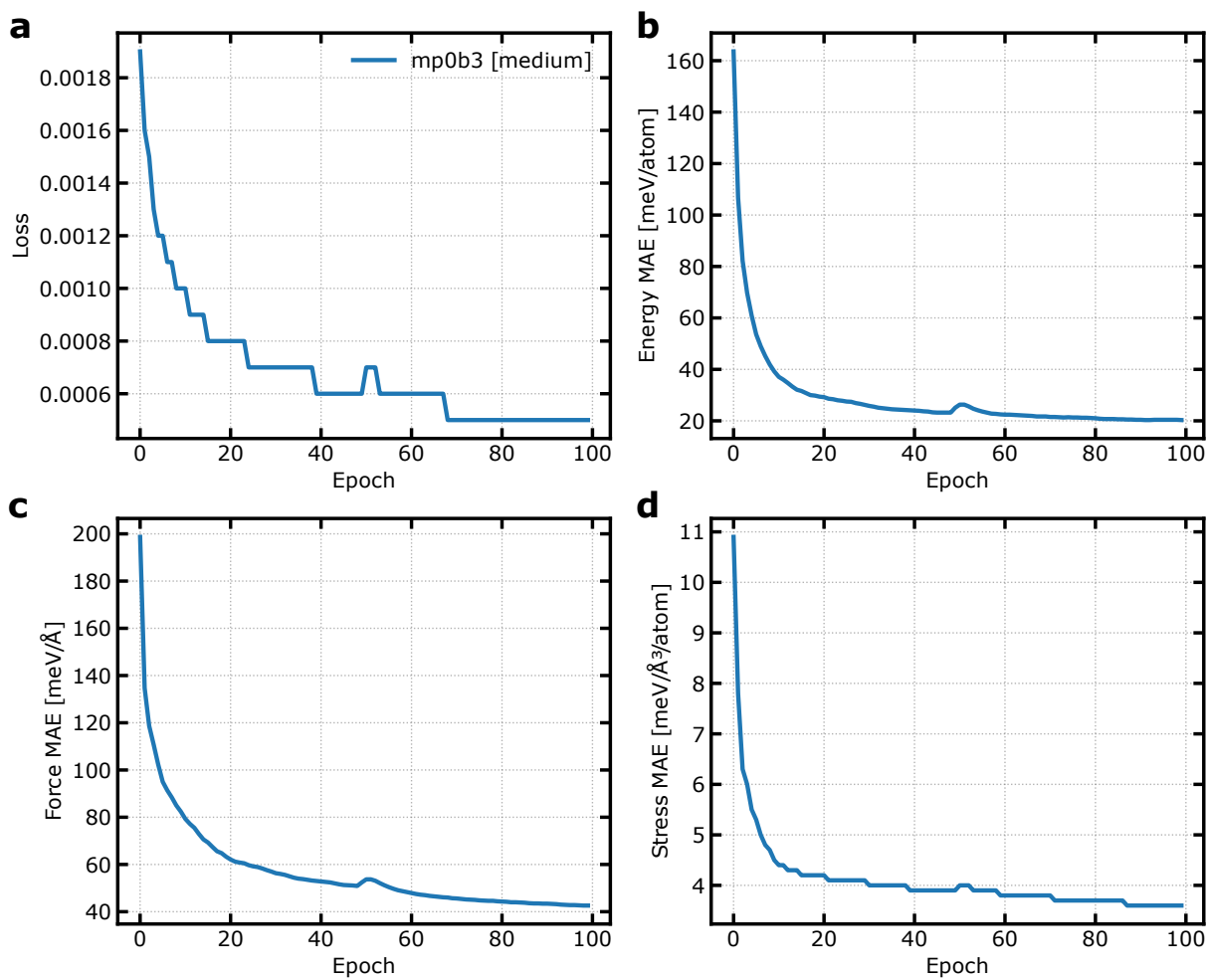


Figure S63: Training curves of MACE-MP-0b3 models fitted to MPtrj data. (a) Loss (eq. (5)). (b-d) Root Mean Square Error (RMSE) of (b) energy per atom, (c) force, and (d) stress per atom. All curves are evaluated on the validation set.

C.2 Fine-tuning protocol

As described in the previous section, MACE-MP-0 offers qualitatively good performance across a wide range of chemistry and materials at the PBE+U level of theory. However, for specific applications, it may be beneficial to fine-tune the model to improve its accuracy. There are two main motivations to fine-tune a foundation model:

1. To reach better quantitative accuracy for a specific application, in order to compute an observable that requires high precision.
2. To increase the electronic-structure level of theory, in case a more accurate functional is required for a specific application.

The fine-tuning process involves training the model on a new dataset, called the fine-tuning dataset, which contains a limited amount of data that are only relevant to the specific application. The goals in designing a good fine-tuning protocol are threefold:

1. To ensure that the fine-tuned model achieves the desired accuracy for the specific application (no under-fitting).
2. To ensure that the fine-tuned model retains the robustness of the foundation model (no catastrophic forgetting).
3. Ensure that the cost of the fine-tuning process is reasonable.

Fine-tuning protocols have received significant attention in the machine learning community (322, 323), especially in the context of computer vision and natural language processing. The most naive protocols consist of just continuing training the model on the new dataset and restarting from the pre-trained model parameters. However, this approach is prone to catastrophic forgetting (91), where the model forgets the knowledge it has learned during the pre-training phase. In the context of ML potentials, this can lead to a model that is not robust and that is more prone to explode during molecular dynamics simulations. To mitigate catastrophic forgetting, several strategies have been proposed that can be decomposed into four main categories: (1) regularization, (2) architecture, (3) data, and (4) optimization.

Today, there exists a wide variety of fine-tuning strategies in the literature that are often specific to the application, the resource constraints, and the desired accuracy. Most state-of-the-art fine-tuning protocols use a combination of regularization, architecture, data, and optimization techniques (324). One of the most widely used approaches is the replay buffer that consists in storing a subset of the pre-training dataset and replaying it during the fine-tuning phase.

In the context of ML potentials, the pre-training dataset is often very large, and efficient subsampling techniques are required to maximize the diversity of the replay buffer while keeping the computational cost reasonable. Moreover, the model might be fine-tuned to new data that were generated using a different level of theory or electronic structure code from the one used to generate the pre-training dataset. Therefore, the model needs to be able to learn from potentially inconsistent data effectively.

In the following, we present a fine-tuning protocol that we call the *multi-head replay* that is specifically designed for ML potentials and that is used to fine-tune the MACE-MP-0 model for different tasks. We demonstrate that fine-tuned models using this protocol achieve the desired accuracy for the specific application using a few configurations, while retaining the robustness of the foundation model.

C.2.1 Multi-head replay fine-tuning protocol

The multihead replay protocol consists of two main steps: (1) the replay buffer construction and (2) the fine-tuning phase. The construction of the replay buffer consists of selecting a subset of the pre-training dataset that will maximize the robustness of the fine-tuned model while keeping the computational cost reasonable.

Let $\mathcal{D}_{\text{pre}}$ be the pre-training dataset, and $\mathcal{D}_{\text{fine}}$ be the fine-tuning dataset. And let Z_{pre} and Z_{fine} be the set of atomic numbers present in $\mathcal{D}_{\text{pre}}$ and $\mathcal{D}_{\text{fine}}$, respectively. We first select a subset of the pre-training dataset that has at least two elements that are present in the fine-tuning dataset,

$$\mathcal{D}_Z = \{d \in \mathcal{D}_{\text{pre}} \mid \exists Z_1, Z_2 \in Z_{\text{fine}}, Z_1 \neq Z_2, Z_1, Z_2 \in d\}. \quad (\text{S12})$$

If the size of $\mathcal{D}_Z$ is smaller than a predefined threshold, we add a random selection of configurations from $\mathcal{D}_{\text{pre}}$, $\mathcal{D}_Z = \mathcal{D}_Z \cup \text{random_sample}(\mathcal{D}_{\text{pre}})$. If the size of $\mathcal{D}_Z$ is larger than a predefined threshold, we subsample it to keep the computational cost reasonable by using a farthest point sampling algorithm. We embed the configurations in a high-dimensional space using the pre-trained model descriptors, and we select the configurations that are the farthest from each other. Note that other subsampling approaches might be more efficient and this is an ongoing research question.

A MACE model can be decomposed into two main parts, the descriptor part and the readout part. The descriptor part is the part of the model that is responsible for computing the atomic features that are invariant to rotations, and the readout maps the atomic features to the atomic energies. In order to train the model on both the pre-training and fine-tuning datasets, we use a multi-head architecture, where the model has two readouts, one for the pre-training dataset and one for the fine-tuning dataset. As the descriptor part is shared between the two readouts, the model can learn from both data sets simultaneously.

An important detail in MACE is the normalization of the total energies. When fitting the energies, we usually subtract the sum of the isolated atomic energies, μ_{Z_a} , to the total energy. This is done to ensure that the model is able to predict the energy of the isolated atoms correctly. As the isolated atomic energies are different for the pre-training and fine-tuning datasets, we need to ensure that the model is able to predict the energy of the isolated atoms correctly for both datasets.

C.2.2 Multi-head fine-tuning examples

r2scan Materials Project dataset One application of fine-tuning is to increase the level of theory of the model. To test the performance of our fine-tuning protocol, we use the r2scan dataset, which is a subset of the Materials Project data set that was recomputed using the r2scan functional. It contains 17,000 relaxed configurations out of the 1.5M configurations in the MPtrj dataset.

Method	Energy MAE (eV)
Scratch ($n_{\text{r2scan}} = 17200$)	0.5430 eV
MP-0 Fine-tune No Replay ($n_{\text{r2scan}} = 17200$)	0.1620 eV
MP-0 Fine-tune Replay + Heads ($n_{\text{r2scan}} = 300$)	0.077 eV
MP-0 Fine-tune Replay + Heads ($n_{\text{r2scan}} = 3000$)	0.039 eV
MP-0 Fine-tune Replay + Heads ($n_{\text{r2scan}} = 17200$)	0.032 eV

Table S8: Comparison of energy MAE for different methods. Models are trained on 17000 configurations from the r2scan Materials Project dataset and test on the 300,000 configurations from the newly discovered Gnome dataset (99). "Scratch" refers to a model trained from scratch on the r2scan dataset. "Fine-tune No Replay" refers to a model fine-tuned just by restarting training.

We observe in Table S8 that going from a model trained from scratch to a model fine-tuned even naively reduces the MAE energy by a factor of 3. Using the multi-head replay protocol with randomly selected 100000 configurations from MPtraj as the replay buffer, we are able to reduce the energy MAE by a factor of 17 compared to training from scratch, reaching an energy MAE of 0.032 eV. The test set consists of the Gnome dataset, which is entirely composed of new crystals.

Fine-tuning on Applications To further demonstrate the effectiveness of our fine-tuning protocols, we pick configuration on a variety of examples of the present paper, and perform fine-tuning on them. For each selected examples, we select a hundred representative configurations and we recompute them using the MPtraj DFT. We then perform a multihead fine-tuning using both the MP-0 and MPA-0 models. We use randomly selected 100000 configurations from MPtraj as the replay buffer, a learning rate of 0.0001 and exponential moving average decay of 0.9999, with Adam optimizer and a batch size of 16. We use a 10% validation set for each task. In order to assess the performance of our fine-tuning protocol, we train models from scratch, using the same model sizes as the foundation models and the default optimization parameters.

In S64, we compare the RMSE forces errors on the validation set of each task, of (i) the two out of the box foundation models (ii) the equivalent models trained from scratch and (iii) the multihead replay fine-tuned

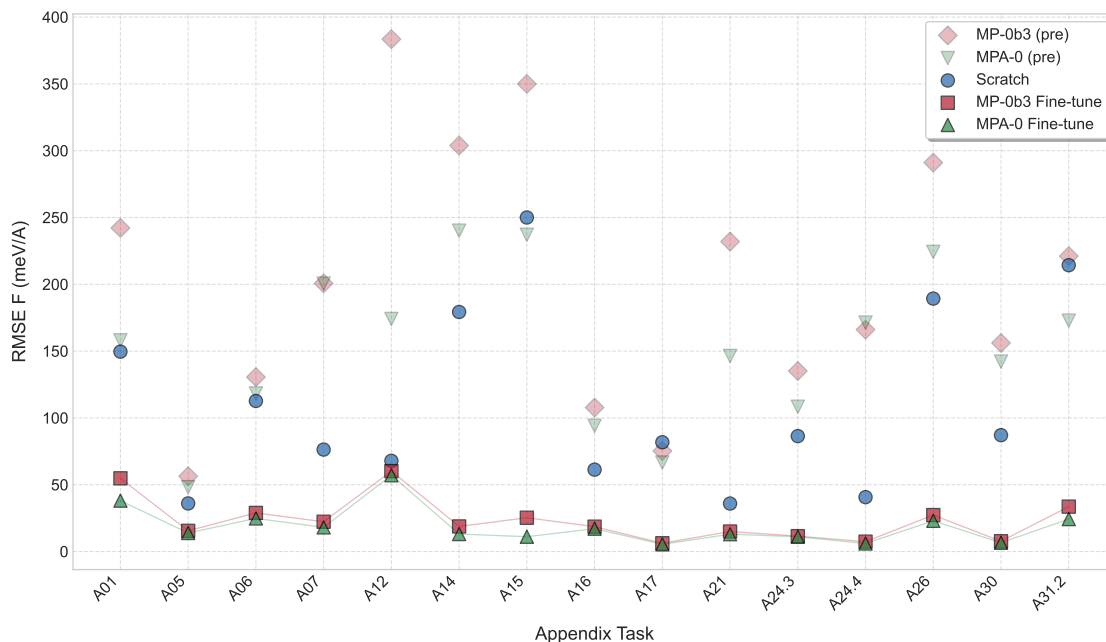


Figure S64: Comparison of validation RMSE on different applications for out of the box foundation models MP-0 and MPA-0, equivalent models trained from scratch and multihead fine-tuned models.

models. The first remarkable result is that the foundation model often matches in performance the from scratch trained model showing the good out of the box performance of these models. The MACE-MPA-0 shows consistent out of the box improvement. Secondly, the fine-tuning shows an improvement of an x4 to x10 in performance compared to the models trained from scratch. Fine-tuning shows impressive data efficiency and reduces significantly the cost of building a system specific model even when the out of the box model is not the most accurate. We also observe only a slight improvement on accuracy for the fine-tuned MPA-0 model compared to MP-0, demonstrating that they are both good for fine-tuning purposes.

To evaluate the amount of forgetting that occurs during fine-tuning, we assess the accuracy of a model fine-tuned on one task (A01) across configurations from other tasks. We then compute the ratio of fine-tuned model accuracy compared to the original model's accuracy. We call this metric the "Forgetting ratio." In Figure S65, we compare the forgetting ratios of two multihead replay fine-tuned models and one naively fine-tuned model. All three models achieve similar validation RMSE on the A01 task and require similar GPU resources for the fine-tuning step. We observe minimal force forgetting in the multihead replay fine-tuned model (forgetting ratios of approximately 1.1 for MPA-0 and 1.4 for MP-0), while the naively fine-tuned model (without replay) shows forgetting ratios around 4.0 for forces. For energy and stress predictions, the multihead replay fine-tuned models demonstrate an order of magnitude improvement in forgetting ratios compared to the naively fine-tuned model. This dramatic reduction in forgetting significantly improves the stability and robustness of the fine-tuned model without compromising accuracy or performance, as all these models maintain similar performance metrics and GPU resource requirements.

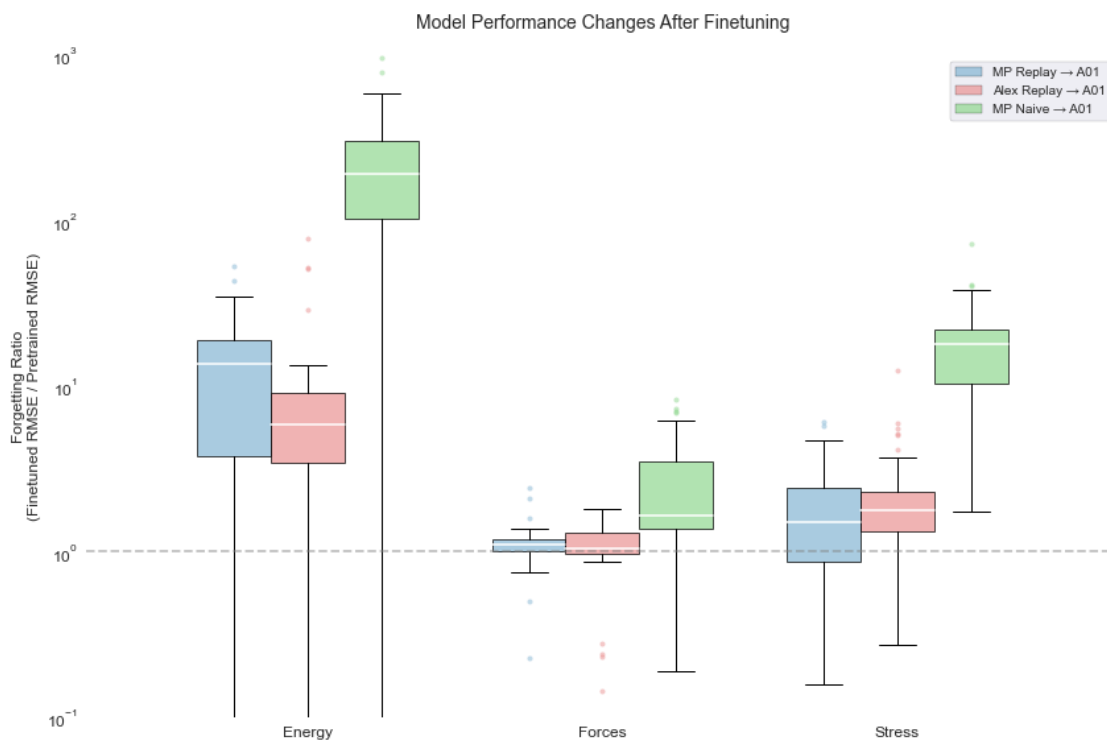


Figure S65: Comparison of performance degradation after fine-tuning across three model variants: MP with multihead replay (blue), Alexandria with multihead replay (pink), and MP with naive fine-tuning (green). The forgetting ratio (y-axis, log scale) represents the ratio of RMSE between fine-tuned and pretrained models for energy, forces, and stress predictions. A ratio of 1.0 (dashed line) indicates no degradation in performance. The box plots show the distribution of ratios across different configurations, with outliers represented as individual points.